This example uses let twice, first to describe a project and then to describe a task. You can then use project and task in the body of the specs. Even though task isn't used in the first spec, that's perfectly fine—RSpec invokes the let block only when the variable is used.

In essence, a let call is syntactic sugar for defining a method and memoizing the result, like this:

```
def me
  @me ||= User.new(name: "Noel")
end
```

The main gotcha here is that the let block isn't executed unless it's invoked. That's often a good thing since your test won't spend time creating unused objects. You can get in trouble sometimes if you expect that the object already exists. For a contrived problem case, note that this example will fail since the two let blocks are never invoked:

```
describe "user behavior"
  let(:me) { User.new(:name => "Noel") }
  let(:you) { User.new(:name => "Erin") }
  specify { User.count.should == 2 }
end
```

Luckily, RSpec provides a mechanism in cases where an item must be present even though it is never invoked by name. It's called let!, with a "bang" for "block always needs getting," which I just made up and only makes a tiny amount of sense, but you probably won't forget it.

One of my other favorite bits of RSpec is an implicit matcher that RSpec creates by name-mangling if you give it a matcher it doesn't recognize. Any matcher of the form be_whatever or be_a_whatever assumes an associated predicate method named whatever?—with a question mark—on the object, and calls that predicate method. The predicate method's return value drives the matcher's behavior. If the matcher is called via to, it passes if the predicate method returns a true value. If the matcher is called via not_to, it passes if the predicate method returns a false value.

In the previous examples, you invoked expectations like expect(project.done?).to be_truthy. Since done? is a predicate method, I rewrote those examples to be more direct. This gives us expect(project).to be_done, which almost reads as simple, clear, natural language. Often it's easier to add a predicate method to your object than it is to create a custom matcher in RSpec.

A lot of complexity goes into making RSpec's language clear. In addition to allowing be_a and be_an, if the predicate method is in the present tense, such